

1 Operators of the CALC Library

Summary

Arithmetic operators

The following operators and expressions can be used to formulate conditions:

Functions	Meaning/ usage
abs(x)	Calculates the absolute value
atan(x)	Calculates the arctangent
cosh(x)	Calculates the hyperbolic cosine
float(x)	Transforms the value into a floating-point number
prime(x)	Produces x if x is a prime or calculates the next prime after x if x is no prime
sqrt(x)	Calculates the square root
acos(x)	Calculates the arccosine
atan2(y,x)	Calculates the arctangent of y/x
exp(x)	Calculates the exponential
log(x)	Calculates the logarithm
sin(x)	Calculates the sine
tan(x)	Calculates the tangent
asin(x)	Calculates the arcsine
cos(x)	Calculates the cosine
int(x)	Transforms the value into an integer
log10(x)	Calculates the logarithm
round(x)	Rounds-up to an integer
round(x,y)	Rounds the value x to y decimal places
sinh(x)	Calculates the hyperbolic sine
tanh(x)	Calculates the hyperbolic tangent
trunc(x)	Reduces the value x to an integer value
trunc(x,y)	Reduces the value x to y decimal places
string(x)	Transforms the value into a string

Logical operators

x and y	The AND operator "and" produces the value 1 (true) if both conditions are true. Otherwise the value 0 (false) will be returned.
---------	---

x or y	The OR operator "or" produces the value 1 (true) if one of the conditions is true. Otherwise the value 0 (false) will be returned.
x like y	The comparison operator "like" produces the value 1 (true) if the first operand corresponds to the pattern of the second operand. Otherwise the value 0 (false) will be returned. The pattern may include the following placeholders: "*" 0 –n any character "?" exactly 1 any character
	→
	→
!x	The logical negation (NOT) operator produces the value 0 (false) if its operand is true (nonzero) and the value 1 (true), if its operand is false (0).
	→
x + y	Addition
x – y	Subtraction
x / y	Division
x * y	Multiplication
x ** y	Calculates x to the power of y
x << y x >> y	The bit-wise shift operators shift their first operand to the left (<<) or to the right (>>) by the number of positions the second operand specifies.
	→
	→
	→
	→
	→
	→
	→
	→
	→
	→
	→
x < y	Produces 1 (true) if the value x is smaller than y, otherwise 0 (false)
x <= y	Produces 1 (true) if the value x is smaller than/ equal to y, otherwise 0 (false)
x > y	Produces 1 (true) if the value x is higher than y, otherwise 0 (false)
x >= y	Produces 1 (true) if the value x is higher than/ equal to y, otherwise 0 (false)
x == y	Produces 1 (true) if the value x is equal to y, otherwise 0 (false)
x != y	Produces 1 (true) if the value x is not equal to y, otherwise 0 (false)
x & y	The bit-wise AND-operator compares each bit of its first operand to the corresponding bit of its second operand. If both bits are 1 the corresponding result bit will be set to 1. Otherwise the corresponding result bit will be set to 0.

$x \wedge y$	The bit-wise EXCLUSIVE-OR-operator compares each bit of its first operand to the corresponding bit of the second operand. If one bit is 0 and the other bit is 1, the corresponding result bit will be set to 1. Otherwise the corresponding result bit will be set to 0.
$x \vee y$	The bit-wise INCLUSIVE-OR-operator compares each bit of the first operand to the corresponding bit of its second operand. If either bits is 1, the corresponding result bit will be set to 1. Otherwise the corresponding result bit will be set to 0.

Constants

Constants	Value
pi	3.141592654
e	2.718281828